

Лабораторная работа № 2.2 «Абстрактные синтаксические деревья»

6 апреля 2026 г.

Ольга Ивенкова, ИУ9-62Б

Цель работы

Целью данной работы является получение навыков составления грамматик и проектирования синтаксических деревьев.

Индивидуальный вариант

Подмножество Рефала-5

```
/* Декартово произведение */
CartProd {
  (t.X e.X) (e.Y) = <CartProd-Bind t.X e.Y> <CartProd (e.X) (e.Y)>;
  /* пусто */ (e.Y) = /* пусто */;
}

CartProd-Bind {
  t.X t.Y e.Y = (t.X t.Y) <CartProd-Bind t.X e.Y>;
  t.X /* пусто */ = /* пусто */;
}

/*
  Прибавление 1 к строковой записи числа:
  123 → 124, 99 → 100, 007 → 008
*/
Inc {
  e.Prefix s.Last, '0123456789' : e.1 s.Last s.Next e.2 = e.Prefix s.Next;
  e.Prefix '9' = <Inc e.Prefix> '0';
  /* пусто */ = '1';
}

/* Функция возвращает уникальные идентификаторы с заданным префиксом */
NextId {
```

```

e.prefix
, <Dg counter>
: {
    s.n = e.prefix <Symb s.n> <Br counter '=' <Add 1 s.n>>;
    /* пусто */ = <Br counter '=' 0> <NextId e.prefix>;
};
}

```

Выражения в языке Рефал состояются из строк в кавычках (одинарных или двойных, в контексте лабораторной работы разница между ними не важна), целых десятичных чисел, идентификаторов, переменных, круглых скобок (...) и угловых скобок < ... >.

В строках допустимы обычные escape-последовательности языка Си: \n, \t, \', \", \\, \xHH.

Идентификаторы начинаются с латинской буквы и состоят из латинских букв, цифр, знаков прочерка _ и дефиса -.

Допустимыми именами функций могут быть любые идентификаторы.

Угловые скобки используются для записи вызова функции, поэтому всегда справа от < должен располагаться идентификатор. В принципе, допустимо <Func рассматривать как цельную лексему.

Имя переменной состоит из индикатора типа (буква s, t или e) и индекса, разделённых знаком .. Индекс может быть либо идентификатором, либо целым числом (корректные имена переменных: s.X1, t.a-b, ошибочные имена переменных: s.1X, e._2, t.-a-).

Скобки в выражениях должны быть согласованны, т.е. выражение вида <F (A> B) недопустимо.

Определение функции состоит из имени и тела функции. Тело функции записывается как перечисление предложений, заключённое в фигурные скобки.

Предложение состоит из левой и правой части и заканчивается на точку с запятой.

Выражение, не содержащее вызовов функций, называется образцовым. Выражение общего вида называется результатным.

Левая часть состоит из образца, к которому приписано ноль или более условий. Условия отделяются от образца запятой и состоят из пары результатного и образцового выражений. Если обозначать P — образцовое выражение, R — результатное, то допустимые левые части будут иметь вид

- «P»,
- «P, R : P»,
- «P, R : P, R : P» и т.д.

Правая часть может иметь два вида:

- = R — возврат значения,

- $R : \{ S; \dots \}$ — так называемый «блок» — вызов безымянной функции.

Ключевые слова и идентификаторы чувствительны к регистру.

Синтаксическое расширение для защиты

```
CartProd {
  (e.X) (e.Y)
  = <Map
    {
      t.X = <Map { t.Y = (t.X t.Y); } e.Y>;
    }
    e.X
  >;
}
```

Реализация

Абстрактный синтаксис

Программа состоит из последовательности определений функций:

Program $\rightarrow$ Def*

Определение функции состоит из имени и тела функции, которое заключено в фигурные скобки:

Def $\rightarrow$ IDENT { Sentence* }

Тело функции представляет собой последовательность предложений:

Sentence $\rightarrow$ LeftPart = RightPart ;

Левая часть предложения состоит из образца и нуля или более условий. Условия отделяются от образца запятой:

LeftPart $\rightarrow$ Pattern (, Result : Pattern)*

Условие представляет собой пару выражений: сначала результатное выражение, затем двоеточие и образцовое выражение:

Condition $\rightarrow$, Result : Pattern

Правая часть предложения может иметь два вида: либо обычный результат, либо блок — результат, за которым следует тело безымянной функции, записанное как последовательность предложений в фигурных скобках:

RightPart $\rightarrow$ Result
| , Result : { Sentence* }

Обычное выражение языка бывает двух видов: образцовое и результатное.

Образцовое выражение — это последовательность элементов, в которой вызовы функций не допускаются:

Pattern -> PatternItem*

Результатное выражение — это последовательность элементов, в которой вызовы функций допустимы:

Result -> ResultItem*

Элемент образцового или результатного выражения может быть строкой, целым числом, идентификатором, переменной, либо вложенным выражением в скобках. В результатном выражении дополнительно допускается вызов функции в угловых скобках:

```
PatternItem -> STRING
| INTEGER
| IDENT
| VAR
| (Pattern)
```

```
ResultItem -> STRING
| INTEGER
| IDENT
| VAR
| (Result)
| <IDENT Result*>
```

Лексическая структура и конкретный синтаксис

Лексическая структура языка включает следующие домены:

STRING -> строковый литерал в одинарных кавычках

INTEGER -> [0-9]+

IDENT -> [A-Za-z][A-Za-z0-9_-]*

VAR -> (?:s|t|e)\.(?:[A-Za-z][A-Za-z0-9_-]*|[0-9]+)(?![A-Za-z0-9_-])

Строковые литералы поддерживают стандартные escape-последовательности:

\n \t \' \" \\ \xnn

Идентификаторы и имена функций чувствительны к регистру.

Имена переменных состоят из индикатора типа s, t или e, затем точки и индекса.

Индекс может быть либо идентификатором, либо целым числом.

Комментарии имеют вид: /*...*/.

Конкретный синтаксис языка:

Program -> Functions

Functions -> e | Function Functions

```

Function -> IDENT '{' Sentences '}'
Sentences -> e | Sentence Sentences
Sentence -> LeftPart RightPart ';'
LeftPart -> Pattern Conditions
Conditions -> e | Condition Conditions
Condition -> ',' Result ':' Pattern
RightPart -> '=' Result
    | ',' Result ':' '{' Sentences '}'
Pattern -> PatternItems
PatternItems -> e | PatternItem PatternItems
PatternItem -> STRING
    | INTEGER
    | IDENT
    | VAR
    | '(' Pattern ')'
Result -> ResultItems
ResultItems -> e | ResultItem ResultItems
ResultItem -> STRING
    | INTEGER
    | IDENT
    | VAR
    | '(' Result ')'
    | '<' IDENT Result '>'

```

Программная реализация

```

from __future__ import annotations

import ast
from dataclasses import dataclass
from abc import ABC
import parser_edsl as pe
import sys
from pprint import pprint

# Program -> Function*
@dataclass
class Program:
    functions: list[FunctionDef]

# Function -> IDENT { Sentence* }
@dataclass
class FunctionDef:
    name: str
    body: list[Sentence]

```

```

# Sentence -> LeftPart RightPart ;
@dataclass
class Sentence:
    left: LeftPart
    right: RightPart

# LeftPart -> Pattern ( Condition* )
@dataclass
class LeftPart:
    pattern: PatternExpr
    conditions: list[Condition]

# Condition -> , Result : Pattern
@dataclass
class Condition:
    result: ResultExpr
    pattern: PatternExpr

class RightPart(ABC):
    pass

# RightPart -> = Result
@dataclass
class ReturnRightPart(RightPart):
    expr: ResultExpr

# RightPart -> , Result : { Sentences }
@dataclass
class BlockRightPart(RightPart):
    expr: ResultExpr
    body: list[Sentence]

class ResultItem(ABC):
    pass

class PatternItem(ABC):
    pass

# Result -> ResultItem*
@dataclass
class ResultExpr:
    items: list[ResultItem]

@dataclass
class Lambda(ResultItem):
    sentences: list[Sentence]

```

```

# Pattern -> PatternItem*
@dataclass
class PatternExpr:
    items: list[PatternItem]

# STRING
@dataclass
class StringLiteral(ResultItem, PatternItem):
    value: str

# INTEGER
@dataclass
class IntLiteral(ResultItem, PatternItem):
    value: int

# IDENT
@dataclass
class IdentLiteral(ResultItem, PatternItem):
    name: str

# VAR
@dataclass
class VarRef(ResultItem, PatternItem):
    kind: str
    index: str | int

# (Result)
@dataclass
class ResultGroup(ResultItem):
    expr: ResultExpr

# (Pattern)
@dataclass
class PatternGroup(PatternItem):
    expr: PatternExpr

# <IDENT Result>
@dataclass
class CallExpr(ResultItem):
    name: str
    args: ResultExpr

# -----

```

```

def parse_string_literal(image: str) -> str:
    return ast.literal_eval(image)

def parse_var_ref(image: str) -> VarRef:
    kind, index_text = image.split('.', 1)
    index: str | int = int(index_text) if index_text.isdigit() \
        else index_text
    return VarRef(kind=kind, index=index)

# ----- Терминалы -----
STRING = pe.Terminal(
    'STRING',
    r'"(?:\\.|[^\\"])*"',
    parse_string_literal,
)
INTEGER = pe.Terminal('INTEGER', r'[0-9]+', int)
IDENT = pe.Terminal('IDENT', r'[A-Za-z][A-Za-z0-9_-]*', str)
VAR = pe.Terminal(
    'VAR',
    r'(?:(s|t|e)\.(?:[A-Za-z][A-Za-z0-9_-]*|[0-9]+)(?![A-Za-z0-9_-])',
    parse_var_ref,
)

# ----- Нетерминалы -----

NProgram = pe.NonTerminal('Program')
NFunctions = pe.NonTerminal('Functions')
NFunction = pe.NonTerminal('Function')
NSentences = pe.NonTerminal('Sentences')
NSentence = pe.NonTerminal('Sentence')
NLeftPart = pe.NonTerminal('LeftPart')
NConditions = pe.NonTerminal('Conditions')
NCondition = pe.NonTerminal('Condition')
NRightPart = pe.NonTerminal('RightPart')

NPatternExpr = pe.NonTerminal('PatternExpr')
NPatternItems = pe.NonTerminal('PatternItems')
NPatternItem = pe.NonTerminal('PatternItem')

NResultExpr = pe.NonTerminal('ResultExpr')
NResultItems = pe.NonTerminal('ResultItems')
NResultItem = pe.NonTerminal('ResultItem')

```



```

# ----- Грамматика -----

# Program -> Functions
NProgram |= NFunctions, Program
# Functions -> e | Function Functions
NFunctions |= lambda: []
NFunctions |= NFunction, NFunctions, lambda f, rest: [f] + rest

# Function -> IDENT '{' Sentences '}'
NFunction |= IDENT, '{', NSentences, '}', FunctionDef

# Sentences -> e | Sentence Sentences
NSentences |= lambda: []
NSentences |= NSentence, NSentences, lambda s, rest: [s] + rest

# Sentence -> LeftPart RightPart ';'
NSentence |= NLeftPart, NRightPart, ';', Sentence

# LeftPart -> PatternExpr Conditions
NLeftPart |= NPatternExpr, NConditions, LeftPart

# Conditions -> e | Condition Conditions
NConditions |= lambda: []
NConditions |= NCondition, NConditions, lambda c, rest: [c] + rest

# Condition -> ',' Result ':' Pattern
NCondition |= ',', NResultExpr, ':', NPatternExpr, Condition

# RightPart -> '=' ResultExpr | ',' ResultExpr ':' '{' Sentences '}'
NRightPart |= '=', NResultExpr, ReturnRightPart
NRightPart |= ',', NResultExpr, ':', '{', \
    NSentences, '}', BlockRightPart

# PatternExpr -> PatternItems
NPatternExpr |= NPatternItems, PatternExpr

# PatternItems -> e | PatternItem PatternItems
NPatternItems |= lambda: []
NPatternItems |= NPatternItem, NPatternItems, \
    lambda item, rest: [item] + rest

# PatternItem
NPatternItem |= STRING, StringLiteral
NPatternItem |= INTEGER, IntLiteral

```

```

NPatternItem |= IDENT, IdentLiteral
NPatternItem |= VAR
NPatternItem |= '(', NPatternExpr, ')', PatternGroup

# ResultExpr -> ResultItems
NResultExpr |= NResultItems, ResultExpr

# ResultItems -> e | ResultItem ResultItems
NResultItems |= lambda: []
NResultItems |= NResultItem, NResultItems, \
    lambda item, rest: [item] + rest

# ResultItem
NResultItem |= STRING, StringLiteral
NResultItem |= INTEGER, IntLiteral
NResultItem |= IDENT, IdentLiteral
NResultItem |= VAR
NResultItem |= '(', NResultExpr, ')', ResultGroup
NResultItem |= '<', IDENT, NResultExpr, '>', CallExpr
NResultItem |= '{', NSentences, '}', Lambda

def build_parser() -> pe.Parser:
    parser = pe.Parser(NProgram, method=pe.EARLEY)
    parser.add_skipped_domain(r'\s+')
    parser.add_skipped_domain(r'\/*\[s\S]*?\*/')
    return parser

if __name__ == '__main__':
    parser = build_parser()

    for filename in sys.argv[1:]:
        try:
            with open(filename, 'r', encoding='utf-8') as f:
                tree = parser.parse(f.read())
                pprint(tree)
        except pe.Error as e:
            print(f'Ошибка {e.pos}: {e.message}')
        except Exception as e:
            print(e)

```

Тестирование

Входные данные

```
/* Декартово произведение */
```

```

CartProd {
  (t.a-b e.X) (e.Y) = <CartProd-Bind t.X e.Y> <CartProd (e.X) (e.Y)>;
  /* пусто */ (e.Y) = /* пусто */;
}

CartProd-Bind {
  t.X t.Y e.Y = (t.X t.Y) <CartProd-Bind t.X e.Y>;
  t.X /* пусто */ = /* пусто */;
}

CartProd {
  (e.X) (e.Y)
    = <Map
      {
        t.X = <Map { t.Y = (t.X t.Y); } e.Y>;
      }
      e.X
    >;
}

/*
  Прибавление 1 к строковой записи числа:
  123 → 124, 99 → 100, 007 → 008
*/
Inc {
  e.Prefix s.Last, '0123456789' : e.1 s.Last s.Next e.2 = e.Prefix s.Next;
  e.Prefix '9' = <Inc e.Prefix> '0';
  /* пусто */ = '1';
}

/* Функция возвращает уникальные идентификаторы с заданным префиксом */
NextId {
  e.prefix
  , <Dg counter>
  : {
    s.n = e.prefix <Symb s.n> <Br counter '=' <Add 1 s.n>>;
    /* пусто */ = <Br counter '=' 0> <NextId e.prefix>;
  };
}

```

Вывод на stdout

```

Program(functions=[FunctionDef(name='CartProd',
  body=[Sentence(left=LeftPart(pattern=PatternExpr(
    items=[PatternGroup(expr=PatternExpr(items=[VarRef(kind='t',
      index='a-b'),

```

```

                                VarRef(kind='e',
                                      index='X')))),
PatternGroup(expr=PatternExpr(items=[VarRef(kind='e',
                                      index='Y'))])),
conditions=[]),
    right=ReturnRightPart(
    expr=ResultExpr(items=[CallExpr(name='CartProd-
Bind',
                                args=ResultExpr(items=[VarRef(kind='t',
                                      index='X'),
                                      VarRef(kind='e',
                                      index='Y'))]),
                                CallExpr(name='CartProd',
                                args=ResultExpr(items=[ResultGroup(
                                expr=ResultExpr(items=[VarRef(kind='e',
                                      index='X'))]),
                                ResultGroup(
                                expr=ResultExpr(items=[VarRef(kind='e',
                                      index='Y'))]))])),
Sentence(left=LeftPart(pattern=PatternExpr(
    items=[PatternGroup(expr=PatternExpr(items=[]),
        PatternGroup(expr=PatternExpr(items=[VarRef(kind='e',
            index='Y'))])),
        conditions=[]),
    right=ReturnRightPart(expr=ResultExpr(items=[]))]),
FunctionDef(name='CartProd-Bind',
    body=[Sentence(left=LeftPart(pattern=PatternExpr(items=[VarRef(kind='t',
            index='X'),
            VarRef(kind='t',
            index='Y'),
            VarRef(kind='e',
            index='Y'))]),
        conditions=[]),
    right=ReturnRightPart(expr=ResultExpr(
    items=[ResultGroup(expr=ResultExpr(items=[VarRef(kind='t',
            index='X'),
            VarRef(kind='t',
            index='Y'))]),
        CallExpr(name='CartProd-Bind',
            args=ResultExpr(items=[VarRef(kind='t',
            index='X'),
            VarRef(kind='e',
            index='Y'))]))]),
Sentence(left=LeftPart(pattern=PatternExpr(items=[VarRef(kind='t',
            index='X')]),
        conditions=[]),

```

```

        right=ReturnRightPart(expr=ResultExpr(items=[]))),
FunctionDef(name='CartProd',
body=[Sentence(left=LeftPart(pattern=PatternExpr(
    items=[PatternGroup(expr=PatternExpr(items=[VarRef(kind='e',
                                                    index='X')])),
        PatternGroup(expr=PatternExpr(items=[VarRef(kind='e',
                                                    index='Y')])))],
        conditions=[]),
    right=ReturnRightPart(expr=ResultExpr(items=[CallExpr(name='Map',
        args=ResultExpr(items=[Lambda(
            sentences=[Sentence(left=LeftPart(
                pattern=PatternExpr(items=[VarRef(kind='t',
                                                    index='X')])),
                conditions=[]),
            right=ReturnRightPart(expr=ResultExpr(items=[CallExpr(name='Map',
args=ResultExpr(items=[Lambda(sentences=[Sentence(
    left=LeftPart(pattern=PatternExpr(items=[VarRef(kind='t',
                                                    index='Y')])),
    conditions=[]),
    right=ReturnRightPart(expr=ResultExpr(
        items=[ResultGroup(expr=ResultExpr(items=[VarRef(kind='t',
                                                    index='X'),
                VarRef(kind='t',
                    index='Y')])))])],
        VarRef(kind='e',
            index='Y')])))])],
            VarRef(kind='e',
                index='X')])))])],
        FunctionDef(name='Inc',
            body=[Sentence(left=LeftPart(
                pattern=PatternExpr(items=[VarRef(kind='e',
                                                    index='Prefix'),
                VarRef(kind='s',
                    index='Last')])),
            conditions=[Condition(result=ResultExpr(
                items=[StringLiteral(value='0123456789')]),
                pattern=PatternExpr(items=[VarRef(kind='e',
                                                    index=1),
                VarRef(kind='s',
                    index='Last'),
                VarRef(kind='s',
                    index='Next'),
                VarRef(kind='e',
                    index=2)]))],
            right=ReturnRightPart(expr=ResultExpr(items=[VarRef(kind='e',

```

```

                                index='Prefix'),
                                VarRef(kind='s',
                                index='Next')))),
Sentence(left=LeftPart(pattern=PatternExpr(items=[VarRef(kind='e',
                                index='Prefix'),
                                StringLiteral(value='9')]]),
                                conditions=[]),
                                right=ReturnRightPart(
                                    expr=ResultExpr(items=[CallExpr(name='Inc',
                                    args=ResultExpr(items=[VarRef(kind='e',
                                    index='Prefix')]]),
                                    StringLiteral(value='0')]])),
Sentence(left=LeftPart(pattern=PatternExpr(items=[]),
                                conditions=[]),
                                right=ReturnRightPart(expr=ResultExpr(items=[StringLiteral(value='1')]]))),
                                FunctionDef(name='NextId',
                                body=[Sentence(left=LeftPart(
                                    pattern=PatternExpr(items=[VarRef(kind='e',
                                    index='prefix')]]),
                                    conditions=[]),
                                    right=BlockRightPart(
                                        expr=ResultExpr(items=[CallExpr(name='Dg',
                                        args=ResultExpr(items=[IdentLiteral(name='counter')]])),
                                        body=[Sentence(left=LeftPart(pattern=PatternExpr(items=[VarRef(kind='s',
                                        index='n')]]),
                                        conditions=[]),
                                        right=ReturnRightPart(expr=ResultExpr(items=[VarRef(kind='e',
                                        index='prefix'),
                                        CallExpr(name='Symb',
                                        args=ResultExpr(items=[VarRef(kind='s',
                                        index='n')]]),
                                        CallExpr(name='Br',
                                        args=ResultExpr(items=[IdentLiteral(name='counter'),
                                        StringLiteral(value='='),
                                        CallExpr(name='Add',
                                        args=ResultExpr(items=[IntLiteral(value=1),
                                        VarRef(kind='s',
                                        index='n')]]))]]))]])),
Sentence(left=LeftPart(pattern=PatternExpr(items=[]),
                                conditions=[]),
                                right=ReturnRightPart(expr=ResultExpr(items=[CallExpr(name='Br',
                                args=ResultExpr(items=[IdentLiteral(name='counter'),
                                StringLiteral(value='='),
                                IntLiteral(value=0)]))],
                                CallExpr(name='NextId',
                                args=ResultExpr(items=[VarRef(kind='e',

```

```
index='prefix')))))))]]))]]))
```

Вывод

В ходе выполнения лабораторной работы была разработана грамматика и спроектировано абстрактное синтаксическое дерево для подмножества языка Рефал-5, учитывающее образцовые и результирующие выражения, переменные с типизированными индексами. Реализованный на основе метода Эрли парсер успешно обрабатывает реальные примеры кода, включая вложенные определения функций. Построенное дерево узлов отражает все ключевые конструкции языка: определения функций, предложения с условиями, вызовы в угловых скобках и группирующие скобки. Полученные навыки позволяют формально описывать синтаксис и автоматизировать разбор нетривиальных языков программирования.